

Αλγόριθμοι Δυκτίων και Πολυπλοκότητα

Πρώτη Σειρά Γραπτών Ασκήσεων

Παναγή Αντρέας
Α.Μ. : 7115 1424 00008

Π.Μ.Σ. - ΑΛΜΑ

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο
Αθήνα, Ελλάδα *

Μάιος 2024



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

* Στοιχειοθέτηση με χρήση $\text{\LaTeX}$

Περιεχόμενα

Άσκηση 1	1
Εκφώνηση	1
Λύση	1
Άσκηση 2	3
Εκφώνηση	3
Λύση	3
Άσκηση 3	4
Εκφώνηση	4
Λύση	4
Άσκηση 4	6
Εκφώνηση	6
Λύση	6
Άνω φράγμα λόγου προσεγγισιμότητας	6
Κάτω φράγμα λόγου προσεγγισιμότητας του αλγορίθμου	7
Άσκηση 5	8
Εκφώνηση	8
Λύση	8
Άσκηση 6	10
Εκφώνηση	10
Λύση	10
Online Greedy Algorithm: List Sceduling	10
Offline Greedy Algorithm: Longest-processing-time-first (LPT)	11
Heuristic Algorithm: Largest Differencing Method	12
FPTAS Algorithm: Melissinos and Pagourtzis	12
Άσκηση 7: Min-Cut Algorithm	14
Εκφώνηση	14
Λύση	14
Άσκηση 1.24	14
Άσκηση 1.25	15
Άσκηση 8: Random Sampling	17
Εκφώνηση	17
Λύση	17
Πρώτη εκτίμηση	17
Δεύτερη εκτίμηση	18
Άσκηση 9: (Capacited Max k-Cut)	19
Εκφώνηση	19
Λύση	19

Άσκηση 1

Εκφώνησηση

- (α) Σχεδιάστε f -προσεγγιστικό αλγόριθμο για το πρόβλημα Cardinality Set Cover, όπου f είναι το μέγιστο πλήθος συνόλων στα οποία μπορεί να ανήκει κάποιο στοιχείο γενικεύοντας κατάλληλα τον αλγόριθμο που χρησιμοποιεί maximal matching για το πρόβλημα (Cardinality) Vertex Cover. Αποδείξτε την ορθότητά του και τον λόγο προσέγγισης που επιτυγχάνει. Βρείτε tight example για τον λόγο προσέγγισης του αλγορίθμου σας.
- (β) Εξηγήστε γιατί ο παραπάνω αλγόριθμος δεν είναι f -προσεγγιστικός για το πρόβλημα Weighted Set Cover.
- (γ) Σχεδιάστε f -προσεγγιστικό αλγόριθμο για το πρόβλημα Weighted Set Cover. Αποδείξτε την ορθότητά του και τον λόγο προσέγγισης που επιτυγχάνει. Βρείτε tight example για τον λόγο προσέγγισης του αλγορίθμου.
- Υπόδειξη: γενικεύστε την ιδέα του degree-weighted αλγορίθμου για το Weighted Vertex Cover. Εναλλακτικά, προσαρμόστε κατάλληλα τον αλγόριθμο της 3ης άσκησης της 3ης σειράς του μαθήματος “Αλγόριθμοι και Πολυπλοκότητα (2023-24)”: https://helios.ntua.gr/pluginfile.php/1928/course/section/12406/ask03_w2023.pdf

Λύση

- (α) Θα σχεδιάσουμε f -προσεγγιστικό αλγόριθμο για το Cardinality Set Cover, όπου f είναι το μέγιστο πλήθος συνόλων στα οποία μπορεί να ανήκει κάποιο στοιχείο ως εξής:
- Αρχικά θα πρέπει να γενικεύσουμε την ιδέα της εύρεσης ενός μεγιστικού ταιριάσματος, θα βρούμε δηλαδή στοιχεία που είναι μεταξύ τους “ανεξάρτητα” και έπειτα θα επιλέξουμε όλα τα σύνολα που τα περιέχουν.

Algorithm 1: f -Approximation Algorithm for Set Cover

Input: Σύμπαν U , Συλλογή υποσυνόλων του σύμπαντος $\mathcal{S}$ τ.ω. κάθε $e \in U$ εμφανίζεται σε f το πολύ σύνολα

Output: Κάλυμμα συνόλων $\mathcal{C} \subseteq \mathcal{S}$

```
1 % Αρχικά θα βρούμε ένα σύνολο από “ανεξάρτητα στοιχεία” γενίκευση της εύρεσης
  “μεγιστικού (όχι μέγιστου) ταιριάσματος”
2 Αρχικοποίησε  $P \leftarrow \emptyset$ 
3 Αρχικοποίησε  $A \leftarrow U$ 
4 while  $A \neq \emptyset$  do
5   |   Επέλεξε αυθαίρετα  $e \in A$ 
6   |    $P \leftarrow P \cup \{e\}$ 
7   |   foreach set  $S \in \mathcal{S} : e \in S$  do
8   |   |    $A \leftarrow A \setminus S$ 
9   |   end
10 end
11
12 % Κατασκευή Καλύμματος Συνόλων
13 Αρχικοποίησε  $\mathcal{C} \leftarrow \emptyset$ ;
14 foreach element  $e \in P$  do
15   |   foreach set  $S \in \mathcal{S} : e \in S$  do
16   |   |    $\mathcal{C} \leftarrow \mathcal{C} \cup \{S\}$ ;
17   |   end
18 end
19 return  $\mathcal{C}$ ;
```

Απόδειξη Ορθότητας:

Αρχικά θα δείξουμε ότι πράγματι ο αλγόριθμος επιλέγει σύνολα που καλύπτουν κάθε στοιχείο του U . Έστω $u \in U$, τότε υπάρχει $s \in P$ και $S' \in \mathcal{C}$ τ.ω. $u, s \in S'$ (γιατί ο αλγόριθμος κατασκευής του P τερματίζει αφού κάθε στοιχείο ανήκει στο P ή σε κάποιο κοινό σύνολο με ένα από τα στοιχεία του P). Επίσης $S' \in \mathcal{C}$ και επομένως συμπεραίνουμε ότι το u καλύπτεται από το σύνολο S' .

Απόδειξη λόγου προσεγγισιμότητας

Αρχικά από κατασκευή του P , καμία επιλογή συνόλων δεν μπορεί να καλύψει δύο ή περισσότερα στοιχεία του P . Επομένως και η βέλτιστη OPT λύση θα περιέχει τουλάχιστον ένα σύνολο για κάθε στοιχείο του P , επομένως $|P| \leq \text{OPT}$.

Για κάθε στοιχείο του P εισάγουμε το πολύ f σύνολα στην οικογένεια $\mathcal{C}$ επομένως: $|\mathcal{C}| \leq f \cdot |P|$.

Συνδυάζουμε τα παραπάνω και προκύπτει ότι $|\mathcal{C}| \leq f \cdot |P| \leq f \cdot \text{OPT} \iff \frac{|\mathcal{C}|}{\text{OPT}} \leq f$.

Tight Example:

Tight example θα μπορούσε να αποτελέσει η άπειρη οικογένεια στιγμιότυπων όπου θεωρούμε το $n \in \mathbb{N}$ να είναι οποιοδήποτε πολλαπλάσιο του $f \in \mathbb{N}$, $U = \{v_1, \dots, v_n\}$ και $\mathcal{S} := \left(\frac{U}{f}\right)$ το οποίο συμβολίζει όλους τους συνδυασμούς f στοιχείων του U .

Ο αλγόριθμος θα κατασκευάσει το σύνολο P που θα αποτελείται από $\frac{n}{f}$ σημεία του U και η οικογένεια $\mathcal{C}$ θα έχει συνολικά $f \cdot \frac{n}{f} = n$ στοιχεία αφού κάθε στοιχείο του P εμφανίζεται σε ακριβώς f σύνολα στο $\mathcal{S}$.

Μια OPT λύση αποτελείται από $\frac{n}{f}$ σύνολα ξένα ανά δύο (υπάρχουν διότι μια επιλογή θα μπορούσε να είναι η κάλυψη των n στοιχείων από σύνολα που περιέχουν τα n σε σειρά σε f -άδες).

Ο λόγος είναι $\frac{n}{f} = f$.

- (6) Για τον αλγόριθμο αυτόν μπορούν να κατασκευαστούν στιγμιότυπα και εκτελέσεις του αλγορίθμου με οσοδήποτε μεγάλο λόγο. Αυτό ισχύει διότι ο αλγόριθμος στον υπολογισμό του “γενικευμένου maximal matching” εκτελεί αυθαίρετες επιλογές στοιχείων οι οποίες μπορεί να εντάξουν σύνολα με οσοδήποτε μικρό βάρος και να αποκλείσουν σύνολα με οσοδήποτε μεγάλο βάρος.

- (γ) Η πιο λεπτομερής ανάλυση της αποδοτικότητας του αλγορίθμου γίνεται στην επόμενη άσκηση. Παρακάτω παρατίθεται ο αλγόριθμος.

Algorithm 2: Greedy Weighted Set Cover($U, \mathcal{S}, w$)

Είσοδος: $\mathcal{S} \subseteq 2^U$, $w : \mathcal{S} \rightarrow \mathbb{R}_+$.

Έξοδος: $C \subseteq \mathcal{S} : \cup_{s \in C} s = U$.

```
1 Αρχικοποίησε  $C \leftarrow \emptyset$ .
2 Όρισε  $f(C) \leftarrow \left| \bigcup_{s \in C} s \right|$ . % Πλήθος στοιχείων ένωσης της οικογένειας
3 while  $f(C) \neq |U|$  do
4   |   Επέλεξε  $s \in \mathcal{S}$  που ελαχιστοποιεί την  $c_s := \frac{w(s)}{[f(S \cup \{s\}) - f(C)]}$ 
5   |    $C \leftarrow C \cup \{s\}$ 
6 return  $C$ 
```

Ένα tight example θα μπορούσε να είναι το εξής:

$U = \{a_1, \dots, a_n\}$

$\forall i \in \{1, \dots, n\}, S_i = \{a_i\}, \text{cost}(S_i) = 1/i$

$S_{n+1} = U, \text{cost}(S_{n+1}) = 1 + \varepsilon$, για $\varepsilon > 0$

$\text{SOL} = H_n, \text{OPT} = 1 + \varepsilon$

Επομένως $\rho(n) > H_n - \varepsilon'$ για οποιοδήποτε ε' .

Άσκηση 2

Εκφώνηση

Αποδείξτε ότι ο λόγος προσέγγισης που επιτυγχάνει ο Greedy αλγόριθμος για το Weighted Set Cover είναι στην πραγματικότητα $H_{|S_{\max}|}$, όπου $S_{\max}$ το σύνολο εισόδου με τη μεγαλύτερη πληθικότητα.

Υπόδειξη: αποδείξτε ότι τα στοιχεία οποιουδήποτε συνόλου $S \in \mathcal{S}$ καλύπτονται από τον Greedy με συνολικό κόστος το πολύ $H_{|S|} \cdot \text{cost}(S)$, δηλαδή $\sum_{e \in S} \text{price}(e) \leq H_{|S|} \cdot \text{cost}(S)$.

Λύση

Θα αναλύσουμε τον αλγόριθμο για το πρόβλημα Weighted Set Cover της πιο πάνω άσκησης **Greedy Weighted Set Cover Algorithm**.

Απόδειξη Λόγου Προσεγγισιμότητας

Για λόγους καλύτερης ανάλυσης της αποδοτικότητας του αλγορίθμου θεωρούμε ότι: όταν ένα σύνολο s εισάγεται το κόστος του ανατίθεται στα νέα στοιχεία που εισάγει στο κάλυμμα, τα οποία χρεώνονται με κόστος c_s . Τέλος το συνολικό κόστος του αλγορίθμου είναι το άθροισμα των κοστών που έχουν ανατεθεί στα στοιχεία του U .

Θεωρούμε ένα τυχόν $s = \{x_k, x_{k-1}, \dots, x_1\} \in C^*$ όπου C^* βέλτιστο κάλυμμα.

Χβτγ θεωρούμε ότι ο αλγόριθμος καλύπτει τα στοιχεία του s με την σειρά $x_k, x_{k-1}, \dots, x_1$.

Στην αρχή της επανάληψης όπου ο αλγόριθμος καλύπτει το στοιχείο x_i του s τουλάχιστον i στοιχεία του s παραμένουν ακάλυπτα, επομένως αν ο αλγόριθμος επέλεγε το σύνολο s σε αυτή την επανάληψη θα πλήρωνε κόστος το πολύ $\frac{w_s}{i}$.

Επομένως ο αλγόριθμος σε αυτή την επανάληψη πληρώνει το πολύ $\frac{x_i}{i}$ για κάθε στοιχείο που καλύπτει (ειδικότερα και το x_i).

Επομένως αθροίζοντας ως προς i το μέγιστο συνολικό άθροισμα που μπορεί να ανατεθεί στα στοιχεία του s είναι $w_s H_{|S|} \leq w_s H_{|S_{\max}|}$.

Αφού το C^* είναι (βέλτιστο) κάλυμμα, αθροίζοντας ως προς $s \in C^*$, έχουμε ότι το συνολικό κόστος του αλγορίθμου είναι το πολύ $\sum_{s \in C^*} w_s H_{|S_{\max}|} = H_{|S_{\max}|} \text{OPT}$.

Άσκηση 3

Εκφώνησηση

Τι λόγο προσέγγισης επιτυγχάνει ο Greedy αλγόριθμος για το Maximum Coverage στις παρακάτω περιπτώσεις:

- (α) Τα στοιχεία έχουν βάρος και επιδιώκουμε τη μεγιστοποίηση του βάρους των καλυπτόμενων στοιχείων (με k σύνολα).
- (β) Τα σύνολα έχουν βάρη και μας δίνεται (αντί για πλήθος συνόλων k) ένα budget B , και επιτρέπεται να επιλέξουμε σύνολα με συνολικό βάρος το πολύ B . Θεωρήστε ότι τα στοιχεία σε αυτή την περίπτωση έχουν όλα το ίδιο βάρος.

Λύση

Θεωρούμε τον εξής Greedy αλγόριθμο για την επίλυση του προβλήματος Maximum Coverage ο οποίος έχει λόγο $(1 - \frac{1}{e})$

Algorithm 3: Greedy Algorithm for Maximum Coverage

Input: A universe of elements U , a collection of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$, and an integer k (the number of sets to select)

Output: A collection $\mathcal{C}$ of selected subsets

```
1  $\mathcal{C} \leftarrow \emptyset$  ;
2  $U_{\text{remaining}} \leftarrow U$  ;
3 for  $i \leftarrow 1$  to  $k$  do
4   if  $U_{\text{remaining}} = \emptyset$  then
5     return  $\mathcal{C}$  ;
6   Choose  $S_j \in \mathcal{S} \setminus \mathcal{C}$  that maximizes  $|S_j \cap U_{\text{remaining}}|$ ;
7    $\mathcal{C} \leftarrow \mathcal{C} \cup \{S_j\}$  ;
8    $U_{\text{remaining}} \leftarrow U_{\text{remaining}} \setminus S_j$  ;
9 return  $\mathcal{C}$  ;
```

(α) *Weighted Maximum Coverage problem*

Αν τα στοιχεία έχουν βάρη τότε λόγος του αλγορίθμου θα μπορούσε να γίνει αυθαίρετα μεγάλος. Θεωρούμε το εξής παράδειγμα $k = 1, U = \{v_1, v_2, v_3\}, \mathcal{S} = \{S_1 := \{v_1, v_2\}, S_2 := \{v_3\}\}, w : U \rightarrow \mathbb{R}$ με $w(v_1) = 0.5, w(v_2) = 0.5, w(v_3) = M, M \in \mathbb{R}$. Ο αλγόριθμος θα επιλέξει το S_1 ενώ η βέλτιστη λύση είναι το S_2 και για τον λόγο ισχύει ότι $\rho \geq M$ για οιοδήποτε $M \in \mathbb{R}$.

Παρόλα αυτά ο greedy αλγόριθμος όπου σε κάθε επανάληψη επιλέγει το σύνολο με το μέγιστο βάρος από μη καλυμμένα στοιχεία πετυχαίνει τον ίδιο λόγο $(1 - 1/e)$ και γενικεύει τον παραπάνω αλγόριθμο

(β) *Budgeted Maximum Coverage (with uniform element weights)*

Ομοίως και σε αυτή την περίπτωση ο απλός greedy αλγόριθμος για το Maximum Coverage μπορεί να αποτύχει αυθαίρετα πολύ.

Θεωρούμε το εξής παράδειγμα με:

$U = \{u_1, u_2, u_3, \dots, u_{2M}\}, B = M \in \mathbb{N}, \mathcal{S} = \{s_0 := \{u_1, u_2\}, s_3 := \{u_3\}, \dots, s_{2M} := \{u_{2M}\}\}$ και $c : \mathcal{S} \rightarrow \mathbb{R}$ με $c(s_0) = M, c(s_i) = 0.5, 3 \leq i \leq 2M$.

Ο αλγόριθμος θα επιλέξει το s_0 με συνολικό πλήθος αντικειμένων 2, οπότε συνολικό βάρος $2w$

όπου w το σταθερό κόστος για κάθε αντικείμενο. Η βέλτιστη λύση θα ήταν η επιλογή των $2M$ συνολικά συνόλων $S_3, S_4, \dots, S_{2M}$ τα οποία συνολικά περιέχουν $2M$ αντικείμενα και συνεπώς δίνουν βάρος $2Mw$.

Άρα καταλήγουμε ότι ο λόγος του αλγορίθμου σε αυτή την οικογένεια παραδειγμάτων είναι $\rho \geq M$.

Αφού τα αντικείμενα έχουν ίσο βάρος και ο στόχος μας είναι να μεγιστοποιήσουμε το βάρος, αρκεί να μεγιστοποιήσουμε το πλήθος των αντικειμένων. Επομένως χθγ μπορούμε να θεωρήσουμε ότι τα βάρη κάθε αντικειμένων είναι ίσα με 1 ή ισοδύναμα ότι δεν υπάρχουν βάρη.

Ο παρακάτω αλγόριθμος μπορεί να επιτύχει λόγο $(1 - 1/e)$:

Algorithm 4: Greedy Budgeted Maximum Coverage (Unweighted Elements)

Input: Σύμπαν U , $\mathcal{S} = \{S_1, \dots, S_m\} \subseteq 2^U$, costs $c_i \geq 0$, budget B

Output: $\mathcal{S}' \subseteq \mathcal{S}$ με total cost $\leq B$ ώστε να καλύπτει όσα περισσότερα στοιχεία γίνεται

```

1 Αρχικοποίησε  $C \leftarrow \emptyset$ ,  $\mathcal{S}' \leftarrow \emptyset$ , cost  $\leftarrow 0$ 
2 while True do
3   Έστω  $S^* \in \mathcal{S} \setminus \mathcal{S}'$  τ.ω. μεγιστοποιεί την ποσότητα
       $\frac{|S_i \setminus C|}{c_i}$  υπό την προϋπόθεση ότι cost  $+ c_i \leq B$ 
4   if no such  $S^*$  exists then
5     return  $\mathcal{S}'$ 
6    $\mathcal{S}' \leftarrow \mathcal{S}' \cup \{S^*\}$ 
7    $C \leftarrow C \cup S^*$ 
8   cost  $\leftarrow$  cost  $+ c_{S^*}$ 
9 return  $\mathcal{S}'$ 

```

Έξτρα: Υπάρχει προσεγγιστικός αλγόριθμος για το *Budgeted Maximum Coverage* με λόγο $1 - 1/e$ όπου δεν χρειάζεται να θεωρήσουμε ότι τα βάρη έχουν όλα ίσες τιμές.

Άσκηση 4

Εκφώνηση

Μελετήστε την προσεγγισιμότητα του προβλήματος Dominating Set: δοθέντος γράφου $G(V, E)$ να βρεθεί ελάχιστης πληθικότητας σύνολο κορυφών $D \subseteq V$, ώστε κάθε κορυφή του $V \setminus D$ να έχει μία τουλάχιστον γειτονική κορυφή στο D . Δώστε άνω και κάτω φράγματα για τον λόγο προσέγγισης συγκρίνοντας με το πρόβλημα Set Cover.

Λύση

Θα δείξω ένα άνω φράγμα στον λόγο προσεγγισιμότητας του προβλήματος και με τον σχεδιασμό ενός greedy προσεγγιστικού αλγορίθμου όσο και εναλλακτικά ως ειδική περίπτωση ενός γεννικότερου προβλήματος.

Έπειτα κατασκευάζοντας αναγωγή από το γνωστό πρόβλημα Set Cover το οποίο έχει βέλτιστο λόγο προσεγγισιμότητας $O(\log n)$ θα δώσουμε το κάτω φράγμα στην προσεγγισιμότητα του προβλήματος.

Άνω φράγμα λόγου προσεγγισιμότητας

Τρόπος 1: Ειδική περίπτωση του Set Cover

Αρχικά μπορούμε να δούμε ότι το Dominating Set αποτελεί ειδική περίπτωση του Set Cover, θεωρώντας ως σύμπαν U το σύνολο των κορυφών του γραφήματος $V(G)$ και ως οικογένεια υποσυνόλων την κλειστή γειτονία κάθε κορυφής $N_G[u]$.

Επομένως ο βέλτιστος λόγος προσεγγισιμότητας $O(\log n)$ του Set Cover αποτελεί άνω φράγμα στο βέλτιστο λόγο προσεγγισιμότητας του Dominating Set.

Τρόπος 2: Κατασκευή προσεγγιστικού αλγορίθμου

Αρχικά θεωρούμε τον παρακάτω greedy αλγόριθμο για την προσέγγιση του προβλήματος.

Algorithm 5: Greedy algorithm for the Dominating Set problem

Result: Ένα Σύνολο Κυριαρχίας D για μη κατευθυνόμενο γράφημα $G = (V, E)$

```
1 Αρχικοποίησε  $D \leftarrow \emptyset$ 
2 Θεώρησε κάθε κορυφή  $v$  ως μη κυριαρχημένη
3 while  $\exists$  μη κυριαρχημένη κορυφή do
4   foreach  $v \in V$  do
5     Υπολόγισε  $w(v) := \#$  μη κυριαρχημένων κορυφών στην κλειστή γειτονία  $N[v]$ 
6   end
7   Έστω  $v^*$  η κορυφή με μέγιστο  $w(v)$ 
8    $D \leftarrow D \cup \{v^*\}$ 
9   Θέσε κάθε κορυφή στην  $N[v^*]$  ως κυριαρχημένη
10 end
11 return  $D$ 
```

Αυτός ο αλγόριθμος πετυχαίνει λόγο προσεγγισιμότητας του προβλήματος $H_{\Delta(G)+1} = O(\log n)$ όπου $n = n(G)$.

Απόδειξη Λόγου Προσεγγισιμότητας:

Για καλύτερη ανάλυση της προσεγγισιμότητας του αλγορίθμου θεωρούμε ότι αντί για κόστος 1 κάθε φορά που εισάγουμε μια κορυφή στο D κατά την εκτέλεση του greedy προσεγγιστικού αλγορίθμου, έχουμε κόστος $1/p$ σε κάθε νέα κορυφή που κυριαρχεί ο αλγόριθμος όπου το p αντιστοιχεί στο πλήθος τέτοιων κορυφών στο εκάστοτε δήμα.

Έστω μια βέλτιστη λύση και S^* ένα Σ.Κ. με βέλτιστο (ελάχιστο) μέγεθος.

Εξ ορισμού, κάθε κορυφή είτε ανήκει στο S^* είτε περιέχει ένα γείτονα σε αυτό.

Θεωρούμε ότι για κάθε κορυφή που ανήκει στο $V(G) \setminus S^*$ ανατίθεται σένα γείτονα της στο S^* που την κυριαρχεί, δημιουργώντας με αυτό τον τρόπο “αστέρια” τα οποία έχουν κέντρο κορυφές του S^* . Αρκεί να δείξουμε ότι (σύμφωνα με την νέα κοστολόγηση) αθροιστικό κόστος κάλυψης κάθε αστεριού είναι $H_{\Delta+1}$.

Θεωρούμε ένα “αστέρι” με κέντρο μια κορυφή $v^* \in S^*$ και έστω ότι ο αλγόριθμος επιλέγει μια νέα κορυφή u .

Το πλήθος των νέων κορυφών που κυριαρχούνται είναι $w(u)$, επομένως αν κάποια από τις κορυφές του αστεριού v κυριαρχείται θα χρεωθεί με κόστος $1/w(u)$.

Από το greedy κριτήριο η u έχει το μέγιστο $w(u)$ και άρα $w(u) \geq w(v^*)$.

Επομένως η v θα χρεωθεί το πολύ $1/w(v^*)$ (έπειτα δεν γίνεται να ξαναχρεωθεί).

Επομένως η πρώτη κορυφή του αστεριού που θα καλυφθεί θα γίνει με κόστος το πολύ $1/(d(v^*) + 1)$.

Με το ίδιο επιχείρημα η δεύτερη κορυφή όταν θα καλυφθεί θα γίνει με κόστος το πολύ $1/d(v^*)$ και γενικότερα ο i οστός κόμβος θα καλυφθεί με κόστος το πολύ $1/(d(v^*) + i - 2)$.

Επομένως το συνολικό αθροιστικό κόστος κάλυψης ενός αστεριού είναι το πολύ:

$$\frac{1}{d(v^*)+1} + \frac{1}{d(v^*)} + \frac{1}{d(v^*)-1} + \dots + \frac{1}{2} + 1 = H_{d(v^*)+1} \leq H_{\Delta+1} < \ln(\Delta) + 2.$$

Επομένως με κάθε τρόπο απο τους δύο καταλήγουμε ότι ένα άνω φράγμα για την προσεγγισιμότητα του αλγορίθμου είναι $O(\log n)$.

Κάτω φράγμα λόγου προσεγγισιμότητας του αλγορίθμου

Θα το δείξουμε εκτελώντας την L -reduction $\text{Set Cover} \leq_L \text{Dominating Set}$.

Έστω $(\mathcal{S}, U)$ ένα στιγμιότυπο του Set Cover με U να αποτελεί το σύμπαν $S = \{S_i : i \in I\}$ (όπου θεωρούμε ότι I και U είναι ξένα).

Κατασκευάζουμε το γράφημα $G = (V, E)$ ως εξής: θέτουμε $V = V(G) := V = I \cup U$ και $E = E(G)$ τ.ω. $\{i, j\} \in E \forall i, j \in I$ καθώς και $\{i, u\} \in E \forall i \in I, u \in S_i$.

Έτσι κατασκευάζουμε ένα split graph G όπου $G[I]$ αποτελεί κλίκια και $G[U]$ ανεξάρτητο σύνολο.

$\Rightarrow$ Αν $C = \{S_i : i \in D\}$ μια εφικτή λύση για το πρόβλημα του Set Cover για κάποιο $D \subseteq I$, τότε το D είναι σύνολο κυριαρχίας για το G μεγέθους $|D| = |C|$.

Πράγματι, για κάθε $u \in U$ υπάρχει $i \in D$ τ.ω. $u \in S_i$ και από κατασκευή τα u και i είναι γειτονικά στο G , επομένως το u κυριαρχείται από το i . Επομένως αφού το D είναι μη κενό, κάθε i είναι γειτονικό με κορυφή στο D .

$\Leftarrow$ Αντίστροφα έστω ότι το D είναι σύνολο κυριαρχίας για το G . Τότε είναι δυνατόν να κατασκευάσουμε ένα άλλο σύνολο κυριαρχίας X τ.ω. $|X| \leq |D|$ και $X \subseteq I$ αντικαθιστώντας κάθε $u \in D \cap U$ από ένα γείτονα $i \in I$ της u .

Επομένως το $C = \{S_i : i \in X\}$ είναι εφικτή λύση για το Set Cover με $|C| = |X| \leq |D|$.

Επομένως δεν μπορούμε να λύσουμε το Dominating Set με καλύτερο λόγο από $O(\log n)$ γιατί έτσι θα λύναμε και το Set Cover με τον ίδιο λόγο.

Συνεπώς το πρόβλημα Dominating Set είναι προσεγγίσιμο με λόγο $\Omega(\log n)$.

Άσκηση 5

Εκφώνηση

- (α) Συμπληρώστε την απόδειξη που θα βρείτε στις διαφάνειες για τον λόγο προσέγγισης $\frac{5}{3}$ για το πρόβλημα Metric TSP_{(s,t)-path}. Εξηγήστε τον ρόλο του όρου $c_{s,t}$ στην ανάλυση καθενός από τους δύο επιμέρους αλγορίθμους.
- (β) Δώστε tight example για τους επιμέρους αλγορίθμους, καθώς και για τον συνολικό αλγόριθμο.

Λύση

- (α) Παρακάτω παρατίθεται αναλυτικά ο αλγόριθμος για την προσέγγιση του TSP με λόγο $\frac{5}{3}$ η οποία αποτελεί παραλλαγή του αλγόριθμου του Χριστοφίδη.

- (1) Construct a minimum spanning tree T of the graph G .
- (2) First, determine the set S of vertices that are of wrong degree in T , i.e., the collection of fixed endpoints of even degree and other vertices of odd degree.
Next, construct a minimum matching M on S .
- (3) Consider the graph that is the union of T and M .
This graph is connected and has two odd-degree vertices.
Find an Eulerian path in the resulting graph (this path traverses each edge exactly once and has the two odd-degree vertices as its endpoints).
- (4) Transform the Eulerian path into a Hamiltonian path $P_{s,t}$ by applying shortcuts.

Σημείωση: Στο paper *Analysis of Christofides' heuristic: Some paths are more difficult than cycles*, όπου πρωτο-παρουσιάστηκε ο (γενικότερος από τον) παραπάνω προσεγγιστικός αλγόριθμος για το πρόβλημα Metric TSP Path, ο συγγραφέας της εργασίας J.A. Hoogeveen αποδεικνύει ότι αν απαιτήσουμε Μονοπάτι Χάμιλτον χωρίς σταθερά άκρα ή με ένα σταθερό άκρο ο λόγος παραμένει $3/2$ (στην περίπτωση μας όμως έχουμε δύο σταθερά άκρα, μια αρχική κορυφή s και μια τελική t και ο λόγος γίνεται $5/3$).

Απόδειξη λόγου $5/3$ του αλγορίθμου:

Έστω $P_{s,t}^*$ το optimal Hamilton (s, t) -path.

Αφού το $P_{s,t}^*$ αποτελεί μονοπάτι τότε αποτελεί και δέντρο. Αφού επίσης περνάει από όλες τις κορυφές αποτελεί Spanning Tree και άρα $c(T) \leq c(P_{s,t}^*)$.

Α.ν.δ.ο. ότι $c(M) \leq \frac{2}{3}c(P_{s,t}^*)$.

Θ.δ.ο. το πολυ-σύνολο Q , που περιέχει τις ακμές που ανήκουν στο MST T συν τις ακμές του Optimal Hamilton (s, t) path, μπορεί να διαμεριστεί σε τρία ξένα σύνολα E_1, E_2, E_3 τα οποία το καθένα δίνει ένα τέλειο ταίριασμα στο σύνολο των περιττού βαθμού κορυφών του T μετά το shortcutting (και επομένως υπάρχει E_i για κάποιο $i = 1, 2, 3$ που έχει κόστος $\leq 1/3 (c(T) + c(P_{s,t}^*)) \leq 2/3 \cdot c(P_{s,t}^*)$).

Πράγματι, κάθε κορυφή του T με λάθος βαθμό έχει περιττό βαθμό στο Q , οπότε το S περιέχει όλες τις κορυφές περιττού βαθμού στο Q . Έστω ότι οι κορυφές στο S είναι $2m$ σε πλήθος. Αριθμού τις κορυφές με την σειρά εμφάνισής τους στο $P_{s,t}^*$. Μετά την αρίθμηση τους οι ακμές του $P_{s,t}^*$ είναι οι ακόλουθες $\{s, 1\}, \{1, 2\}, \dots, \{n-2, t\}$, όπου n το πλήθος των κορυφών.

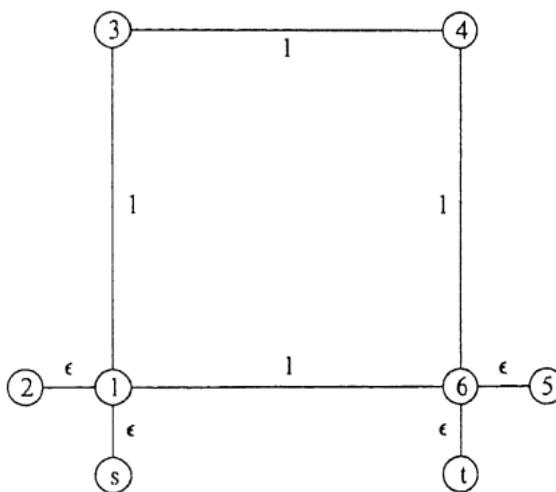
Το πρώτο υποσύνολο E_1 περιέχει τις ακμές μεταξύ των των περιέχει τις ακμές μεταξύ των κορυφών $(2k-1)$ και $(2k)$ στο S για $k = 1, \dots, m$ (δηλ $\{s, 1\}, \{1, 2\}, \dots$).

Το πρώτο τέλειο ταίριασμα το παίρνουμε από shortcutting στο E_1 .

Θεωρούμε το γράφημα που ενάγεται από τις κορυφές $\{s, 1, \dots, n-2, t\}$ και τις ακμές $Q \setminus E_1$.

Το γράφημα είναι συνεκτικό (αφού περιέχει το T) και περιέχει όλες τις κορυφές άρτιου βαθμού λόγω της αφαίρεσης του E_1 , επομένως περιέχει κύκλο Euler. Αυτός ο κύκλος μπορεί να διαμεριστεί σε δύο σύνολα E_2, E_3 . Εκτελώντας shortcutting παίρνουμε δύο τέλεια ταιριάσματα στο S .

(6) Tight example αποτελεί το ακόλουθο:



Σε αυτό το παράδειγμα $0 < \epsilon \ll 1$, και κάθε ακμή που δεν είναι ζωγραφισμένη ισούται με το βάρος του ελάχιστου μονοπατιού μεταξύ των άκρων της.

Το βέλτιστο Χαμιλτονιανό μονοπάτι είναι $P_{s,t}^* \{ \{s, 1\}, \{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 5\}, \{5, 6\}, \{6, t\} \}$, μήκους $3 + 6\epsilon$.

Ένα minimum spanning tree είναι $T = \{ \{s, 1\}, \{1, 2\}, \{1, 3\}, \{1, 6\}, \{4, 6\}, \{5, 6\}, \{6, t\} \}$, συνολικού βάρους $3 + 4\epsilon$.

Το σύνολο των κορυφών λάθος βάρους στο T είναι οι $\{2, 3, 4, 5\}$.

Ένα optimal perfect matching σε αυτές τις κορυφές είναι $M = \{ \{2, 3\}, \{4, 5\} \}$, μήκους $2 + 2\epsilon$.

Ένα Eulerian path στο (πολυ)γράφημα των T και M είναι $\{ \{s, 1\}, \{1, 3\}, \{3, 2\}, \{2, 1\}, \{1, 6\}, \{6, 4\}, \{4, 5\}, \{5, 6\}, \{6, t\} \}$.

Χρησιμοποιώντας shortcuts, το Hamiltonian path που προκύπτει είναι το

$P_{s,t} = \{ \{s, 1\}, \{1, 3\}, \{3, 2\}, \{2, 5\}, \{5, 4\}, \{4, 6\}, \{6, t\} \}$ μήκους $5 + 6\epsilon$.

Θεωρώντας αρκετά μικρό $\epsilon > 0$ κοντά στο μηδέν παίρνουμε λόγο αυδαίρετα κοντά στο $5/3$.

Άσκηση 6

Εκφώνηση

- (α) Αναπτύξτε έναν προσεγγιστικό αλγόριθμο, με όσο το δυνατόν καλύτερο λόγο προσέγγισης για το πρόβλημα Subset Similarity: δίνεται ένα σύνολο A θετικών ακεραίων και ζητείται η διαμέρισή του σε 2 σύνολα A_1, A_2 με όσο το δυνατόν πλησιέστερα αθροίσματα στοιχείων. Δηλαδή θέλουμε να ελαχιστοποιήσουμε τον λόγο $\max(\sum A_1, \sum A_2) / \min(\sum A_1, \sum A_2)$, όπου $\sum A_i$ το άθροισμα των στοιχείων του A_i .
Εξηγήστε την ορθότητα και την πολυπλοκότητα του αλγορίθμου σας.
- (β) [bonus] Σχεδιάστε αλγόριθμο για το πρόβλημα 3-Subset Similarity που έχει την ίδια είσοδο με το Subset Similarity και ζητείται η διαμέριση του A σε 3 σύνολα A_1, A_2, A_3 που να ελαχιστοποιούν τον λόγο $\max(\sum A_1, \sum A_2, \sum A_3) / \min(\sum A_1, \sum A_2, \sum A_3)$

Λύση

Παρακάτω παρατίθενται αλγόριθμοι στην γενική τους μορφή για το k -Subset Similarity όπου μπορούμε να θεωρήσουμε τα Subset Similarity και 3-Subset Similarity ως ειδικές περιπτώσεις (με $k = 2$ και $k = 3$ αντίστοιχα).

Στην αρχή παρουσιάζουμε δύο Greedy αλγορίθμους ένα online και ένα offline και έπειτα δύο άλλους αξιοσημείωτους αλγορίθμους.

Η άσκηση αυτή θα μπορούσε να ενισχυθεί και να περιέχει τις (απαιτητικές) αποδείξεις του λόγου των αλγορίθμων μαζί με τα tight examples.

Online Greedy Algorithm: List Scheduling

Θα παρουσιάσουμε τον γενικότερο αλγόριθμο όπου μοιράζει το σύνολο σε k υποσύνολα.

Algorithm 6: List Scheduling for k Partitions

Input : A list of numbers $A = \{a_1, a_2, \dots, a_n\}$; a positive integer k

Output: A partition $\{P_1, P_2, \dots, P_k\}$ of A such that the maximum subset sum is at most $(2 - \frac{1}{k})$ times optimal

```
1 for  $i \leftarrow 1$  to  $k$  do
2    $P_i \leftarrow \emptyset$ 
3    $S_i \leftarrow 0$ 
   // Initialize subset  $P_i$  and its sum  $S_i$ 
4 for each element  $a \in A$  do
5    $j \leftarrow \operatorname{argmin}_{1 \leq i \leq k} S_i$ 
   // Find the subset with the smallest current sum  $P_j \leftarrow P_j \cup \{a\}$ 
6    $S_j \leftarrow S_j + a$ 
7 return  $\{P_1, P_2, \dots, P_k\}$ 
```

Λόγος Αλγορίθμου:

Ο αλγόριθμος αυτός πετυχαίνει λόγο $2 - \frac{1}{k}$.

Ειδικότερα για $k = 2$ και $k = 3$ ο αλγόριθμος αυτό λύνει το πρόβλημα 2-Subset Similarity με λόγο $\frac{3}{2} = (2 - \frac{1}{2})$ και το πρόβλημα 3-Subset Similarity με λόγο $\frac{5}{3}$.

Ο λόγος αυτός είναι tight.

Απόδειξη Λόγου:

Έστω $a_1, \dots, a_n$ οι αριθμοί και k το πλήθος των συνόλων.

Αρχικά, έστω $s_{\text{ελ}}$ το ελάχιστο από τα άθροισματα, τότε ισχύει ότι $t_{\text{ελ}} \leq \frac{\sum_{i=1}^n a_i}{k}$.

Επίσης $OPT \geq \frac{\sum_{i=1}^n a_i}{k}$ και $OPT \geq \max_i \{a_i\} (*)$.

Έστω l το σύνολο με το μεγαλύτερο άθροισμα, έστω a_l το τελευταίο στοιχείο που ανατέθηκε στο σύνολο αυτό και έστω t_l το άθροισμα του συνόλου πριν ανατεθεί το s_l σε αυτό.

Προφανώς για να ανατεθεί το στοιχείο a_l στο l ίσχυε ότι το l είχε το μικρότερο τρέχον άθροισμα (**).

Επομένως έχουμε:

$$s_l + \frac{a_l}{k} = \frac{k \cdot s_l + a_l}{k} \stackrel{(**)}{\leq} \frac{\sum_{i=1}^n a_i}{k} \leq OPT$$

Επομένως $s_l \leq OPT - \frac{a_l}{k} \leq OPT(1 - \frac{1}{k})$ λόγω της (*).

Επομένως το μέγιστο άθροισμα για το μέγιστο άθροισμα T ισχύει ότι:

$$\begin{aligned} T &= s_l + a_l \\ &\leq (1 - 1/k) \cdot OPT + OPT \\ &= (2 - 1/k) \cdot OPT \end{aligned}$$

Αξίζει επίσης να σημειωθεί ότι:

1. Μπορούμε να εφαρμόσουμε αυτό τον αλγόριθμο ως online αλγόριθμο, δηλαδή και στις περιπτώσεις που δεν μπορούμε να γνωρίζουμε το μέλλον πάρα μόνο την εκάστοτε νέα δοθέν είσοδο
2. Ο χρόνος του αλγορίθμου είναι γραμμικός $O(n)$

Offline Greedy Algorithm: Longest-processing-time-first (LPT)

Η ιδέα του αλγορίθμου αυτού είναι ίδια με του List Scheduling Algorithm και εντάσσεται σε μια πιο γενική οικογένεια αλγορίθμων που θεωρούν την είσοδο γνωστή εξ αρχής, δίνοντάς τους την δυνατότητα να εκτελέσουν μια προεργασία σε αυτήν πριν εφαρμόσουν την ιδέα του List Scheduling.

Algorithm 7: Longest Processing Time First (LPT) for Number Partitioning

Input: A set of numbers $S = \{s_1, s_2, \dots, s_n\}$

Output: Two subsets A and B which approximately balance the total sum

```

1 Sort  $S$  in non-increasing order;
2 Initialize  $A \leftarrow \emptyset, B \leftarrow \emptyset$ ;
3 Initialize  $sum_A \leftarrow 0, sum_B \leftarrow 0$ ;
4 for each element  $s \in S$  do
5   if  $sum_A \leq sum_B$  then
6      $A \leftarrow A \cup \{s\}$ ;
7      $sum_A \leftarrow sum_A + s$ ;
8   else
9      $B \leftarrow B \cup \{s\}$ ;
10     $sum_B \leftarrow sum_B + s$ ;
11 return  $(A, B)$ ;
```

Αξίζει να σημειωθεί ότι:

1. Ο αλγόριθμος αυτός θεωρείτε online καθώς θεωρούμε εξ αρχής γνωστή όλη την είσοδο ώστε να μπορούμε να εκτελέσουμε μια προεργασία σε αυτήν (σε αυτή την περίπτωση sorting)
2. Ο χρόνος του αλγορίθμου είναι (μεγαλύτερος από του (απλού) List Scheduling) $O(n \log n)$
3. Ο λόγος του αλγορίθμου είναι $\frac{4k-1}{3k}$ και είναι tight (για $k = 2$ έχουμε λόγο $\frac{7}{6}$ και για $k = 3$ παίρνουμε $\frac{11}{9}$)

Heuristic Algorithm: Largest Differencing Method

Ο αλγόριθμος αυτός καλείται επίσης: Karmarkar–Karp Algorithm.

1. Order the numbers from large to small.
2. Replace the largest and second-largest numbers by their difference.
3. If two or more numbers remain, return to step 1.
4. Using backtracking, compute the partition.

Ο αλγόριθμος αυτό δίνει στην χειρότερη $\frac{4}{3} - \frac{1}{3k}$ όπου για $k = 2, 3$ έχουμε $\frac{7}{6}$ και $\frac{11}{9}$.

Σημείωση: Υπάρχουν πολλές παραλλαγές του αλγορίθμου: PDM (Paired Differencing Method), RLDM (Restricted Largest Differencing Method), BLDM (Balanced Largest Differencing Method).

FPTAS Algorithm: Melissinos and Pagourtzis

Παρουσιάζεται (ελπίξω) περιληπτικά η κεντρική ιδέα της εργασίας.

Αρχικά δοθέντος ενός συνόλου $A = \{a_1, \dots, a_n\}$ από n θετικούς ακέραιους και δύο συνόλων $S_1, S_2 \subseteq \{1, \dots, n\}$ ορίζουμε ως $\mathcal{MR}$ της εξής συνάρτησης:

$$\mathcal{MR}(S_1, S_2, A) := \begin{cases} \max \left\{ \frac{\sum_{i \in S_1} a_i}{\sum_{j \in S_2} a_j}, \frac{\sum_{j \in S_2} a_j}{\sum_{i \in S_1} a_i} \right\} & \text{if } S_1 \cup S_2 \neq \emptyset \\ \infty & \text{otherwise} \end{cases}$$

Επομένως δίνουμε τον ισοδύναμο ορισμό:

Subset-Sums Ratio problem (SSR).

Δοθέντος ενός συνόλου $A = \{a_1, \dots, a_n\}$ και n θετικούς ακέραιους, βρές δύο ξένα σύνολα $S_1, S_2 \subseteq \{1, \dots, n\}$ τ.ω. $\mathcal{MR}(S_1, S_2, A)$ ελαχιστοποιείται.

Θα δώσουμε ακόμα ένα ισοδύναμο ορισμό του προβλήματος μέσω μιας παραλλαγής:

Semi-Restricted Subset-Sums Ratio problem.

Δοθέντος ενός συνόλου $A = \{a_1, \dots, a_n\}$ από n θετικούς ακέραιους και ακέραιο $1 \leq p < n$, βρες δύο ξένα σύνολα $S_1, S_2 \subseteq \{1, \dots, n\}$ τ.ω. $\max S_1 = p < \max S_2$ και η τιμή $\mathcal{MR}(S_1, S_2, A)$ ελαχιστοποιείται.

Συνεπώς, παρατηρούμε ότι:

Αν $A = \{a_1, \dots, a_n\}$ ένα σύνολο από n θετικούς ακεραίους και $p \in \{1, \dots, n-1\}$.

Αν S_1^*, S_2^* η optimal solution του SSR problem για ένα στιγμιότυπο A και S_1^p, S_2^p η optimal solution του Semi-Restricted SSR problem για το στιγμιότυπο A, p τότε:

$$\mathcal{MR}(S_1^*, S_2^*, A) = \min_{p \in \{1, \dots, n-1\}} \mathcal{MR}(S_1^p, S_2^p, A)$$

Επομένως για να βρούμε βέλτιστη λύση στο SSR πρόβλημα αρκεί να λύσουμε το Semi-Restricted SSR πρόβλημα για όλα τα $p \in \{1, \dots, n-1\}$.

Η λύση FPTAS λύση του προβλήματος πηγάζει από την ακριβή ψευδοπολυωνυμική λύση του προβλήματος.

Παρακάτω παρατίθεται ο $(1 + \varepsilon)$ προσεγγιστικός FPTAS αλγόριθμος.

Algorithm 8: FPTAS for Semi-Restricted SSR — $\text{SOL}_{\text{apx}}(A, p, \varepsilon)$ function

Input: A strictly sorted set $A = \{a_1, \dots, a_n\}$, $a_i \in \mathbb{Z}^+$, an integer p with $1 \leq p < n$, and an error parameter $\varepsilon \in (0, 1)$.

Output: The sets of a $(1 + \varepsilon)$ -approximation solution for Semi-Restricted SSR.

```
1  $\delta \leftarrow \frac{\varepsilon \cdot a_p}{3n}$ ;
2  $A' \leftarrow \emptyset$ ;
3 for  $i \leftarrow 1$  to  $n$  do
4    $a'_i \leftarrow \lfloor \frac{a_i}{\delta} \rfloor$ ;
5    $A' \leftarrow A' \cup \{a'_i\}$ ;
6  $(S_1, S_2) \leftarrow \text{SOL}_{\text{ex}}(A', p)$ ;
7 return  $(S_1, S_2)$ ;
```

Όσο αφορά την ακριβή λύση παρατηρούμε τα παρακάτω και διακρίνουμε τις εξής δύο περιπτώσεις:
Έστω A, p ένα στιγμιότυπο του Semi-Restricted SSR problem όπου $A = \{a_1, \dots, a_n\}$ και $1 \leq p < n$.
Για να λύσουμε το βλήμα ελέγχουμε δύο περιπτώσεις για το μέγιστο στοιχείο της optimal λύσης.
Έστω S_1^*, S_2^* μια βέλτιστη λύση για αυτό το στιγμιότυπο και ότι $\max S_2^* = q$.
Ορίζουμε $B = \{a_i | i > p, a_i < p, \sum_{j=1}^p a_j\}$ και $C = \{a_i | a_i \geq \sum_{j=1}^p a_j\}$ όπου είτε $a_q \in B$ είτε $a_q \in C$.
Σημειώνουμε ότι $A = \{a_1, \dots, a_p\} \cup B \cup C$.

Η ακριβής λύση υπολογίζεται με δυναμικό προγραμματισμό παίρνοντας περιπτώσεις $a_q \in B$ είτε $a_q \in C$.

Άσκηση 7: Min-Cut Algorithm

Εκφώνηση

Να λύσετε τις Ασκήσεις 1.24 και 1.25 του M. Mitzenmacher and E. Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.

Άσκηση 1.24:

Generalizing on the notion of a cut-set, we define an r -way cut-set in a graph as a set of edges whose removal breaks the graph into r or more connected components. Explain how the randomized min-cut algorithm can be used to find minimum r -way cut-sets, and bound the probability that it succeeds in one iteration.

Άσκηση 1.25:

To improve the probability of success of the randomized min-cut algorithm, it can be run multiple times.

- Consider running the algorithm twice. Determine the number of edge contractions and bound the probability of finding a min-cut.
- Consider the following variation. Starting with a graph with n vertices, first contract the graph down to k vertices using the randomized min-cut algorithm. Make copies of the graph with k vertices, and now run the randomized algorithm on this reduced graph l times, independently. Determine the number of edge contractions and bound the probability of finding a minimum cut.
- Find optimal (or at least near-optimal) values of k and l for the variation in (b) that maximize the probability of finding a minimum cut while using the same number of edge contractions as running the original algorithm twice.

Λύση

Άσκηση 1.24

Γενικεύοντας την ιδέα του αλγορίθμου του min cut καταλήγουμε στον εξής αλγόριθμο.

Algorithm 9: Randomized r Way Min Cut

Input: An undirected connected graph $G = (V, E)$, integer $r \geq 2$

Output: An r way cutset $C \subseteq E$

```
1  $H := (V_H, V_E) \leftarrow (V, E);$ 
2 while  $|V_H| > r$  do
3   Choose an edge  $e \in V_E$  uniformly at random;
4   Contract  $e$  in  $H$ : merge its endpoints into a single super vertex;
5   Remove any self loops from  $V_E$ ;
6 Let  $C \leftarrow \{ \{u, v\} \in E \mid u, v \text{ lie in two different super vertices of } H \};$ 
7 return  $C$ ;
```

Ομοίως παρατηρούμε ότι κάθε r -way cut-set του γραφήματος σε κάθε επανάληψη αποτελεί και r -way cut-set του αρχικού.

Κάτω φράγμα της πιθανότητας του αλγορίθμου να επιστρέψει ορθό αποτέλεσμα.

Σε ένα στάδιο όπου απομένουν i υπερκορυφές, αν επιλέξουμε τυχαία ένα σύνολο $r - 1$ κορυφών,

η πιθανότητα μία δεδομένη ακμή να μην ανήκει στο τυχαίο r -way cut-set που ορίζουν οι κορυφές αυτές είναι:

$$\Pr[\text{η ακμή να μην ανήκει στο τυχαίο cut-set}] = \frac{\binom{i-2}{r-1}}{\binom{i}{r-1}} = \frac{(i-r+1)(i-r)}{i(i-1)}.$$

Αυτή η φόρμουλα προκύπτει από απλή καταμέτρηση των τρόπων επιλογής των $r-1$ κορυφών, όπου αποφεύγουμε τις δύο άκρες της εξεταζόμενης ακμής.

Έστω C ένα ελάχιστο σύνολο ακμών που προκαλεί r συνεκτικά συστατικά. Επειδή C είναι ελάχιστο, δεν μπορεί να είναι μεγαλύτερο από το αναμενόμενο μέγεθος της τυχαίας τομής που αναφέραμε παραπάνω. Άρα

$$|C| \leq \left(1 - \frac{(i-r+1)(i-r)}{i(i-1)}\right) |E| \implies 1 - \frac{|C|}{|E|} \geq \frac{(i-r+1)(i-r)}{i(i-1)}.$$

Αυτή η ανισότητα χρησιμοποιείται για να φράξει το μέγεθος του C σε σχέση με το πλήθος των ακμών.

Από $i = n$ έως $i = r+1$, η συνολική πιθανότητα καμία από τις $n-r$ τυχαίες συγχωνεύσεις να μην συνθλίψει ακμή του C είναι τουλάχιστον:

$$\prod_{j=0}^{n-r-1} \frac{(n-r+1-j)(n-r-j)}{(n-j)(n-j-1)} = \frac{r!(r-1)!(n-r)!(n-r+1)!}{n!(n-1)!},$$

η οποία είναι ασυμπτωτικά $\Omega(n^{-2(r-1)})$.

Σημείωση: Παρατηρούμε ότι πράγματι η λύση για το πρόβλημα του Min Cut αποτελεί ειδική περίπτωση για $r=2$ και όσο αφορά τον ορισμό του προβλήματος και όσο αφορά την λύση του.

Άσκηση 1.25

- (a) Σε κάθε εκτέλεση ο αλγόριθμος Karger Min Cut πραγματοποιεί $n-2$ τυχαίες συγχωνεύσεις. Για δύο ανεξάρτητες εκτελέσεις, ο συνολικός αριθμός συγχωνεύσεων είναι: $2(n-2)$.

Η πιθανότητα να διατηρηθεί η πραγματική ελάχιστη τομή σε μετά από όλες τις $n-2$ συγχωνεύσεις (και επομένως να βρεθεί η τομή) είναι

$$p := \prod_{i=0}^{n-3} \frac{n-2-i}{n-i} = \frac{2}{n(n-1)}.$$

Έστω $p = \frac{2}{n(n-1)}$ η πιθανότητα επιτυχίας σε μία εκτέλεση. Τότε η πιθανότητα να επιτύχουμε τουλάχιστον μία φορά σε δύο ανεξάρτητα τρεξίματα είναι το συμπλήρωμα της πιθανότητας αποτυχίας και των δύο φορές:

$$1 - (1-p)^2 = 1 - \left(1 - \frac{2}{n(n-1)}\right)^2 = \dots = \frac{4}{n(n-1)} - \frac{4}{n^2(n-1)^2}$$

- (b) Αρχικά, εκτελούμε $n-k$ τυχαίες συγχωνεύσεις στο αρχικό γράφημα με n κορυφές, ώστε να απομείνουν k υπερκορυφές.

Έπειτα, δημιουργούμε l ανεξάρτητ αντίγραφα του γραφήματος με k κορυφές.

Σε κάθε αντίγραφο τρέχουμε τον αλγόριθμο Karger μέχρι να μείνουν 2 κορυφές, πραγματοποιώντας $k-2$ συγχωνεύσεις.

Συνολικός αριθμός συγχωνεύσεων:

$$(n - k) + l(k - 2).$$

Πιθανότητα επιτυχίας πρώτου σταδίου ($n \rightarrow k$):

$$p_1 \geq \frac{\binom{k}{2}}{\binom{n}{2}} = \frac{k(k-1)}{n(n-1)}.$$

Εντοπισμός τομής σε μία αντιγραφή ($k \rightarrow 2$):

$$p_2 \geq \frac{2}{k(k-1)}.$$

Συνολική πιθανότητα με l τρεξίματα:

$$p_{\text{ολ}} \geq p_1 [1 - (1 - p_2)^l] = \frac{k(k-1)}{n(n-1)} \left[1 - \left(1 - \frac{2}{k(k-1)} \right)^l \right].$$

- (c) Αφού οι το συνολικό πλήθος συνδλίψεων θα πρέπει να είναι ίσο με το πλήθος που εκτελούνται σε δύο απλές επαναλήψεις του απλού Min-Cut αλγορίθμου, τότε:

$$n - k = l(k - 2) = 2(n - 2) \iff l = \frac{n + k - 4}{k - 2}$$

Θεωρούμε την

$$f(k) := \frac{k(k-1)}{n(n-1)} \left[1 - \left(1 - \frac{2}{k(k-1)} \right)^{l(k)} \right].$$

Και ψάχνουμε ολικό μέγιστο σε αυτήν ως προς το k .

Άσκηση 8: Random Sampling

Εκφώνηση

Θεωρούμε μια δημοσκόπηση για τη στάση των πολιτών ως προς μια σημαντική πολιτικο-οικονομική μεταβολή. Οι πολίτες απαντούν στη δημοσκόπηση με “ναι” ή “όχι” (υπέρ ή εναντίον της μεταβολής). Αν το (πραγματικό) ποσοστό των πολιτών που τάσσονται υπέρ της μεταβολής είναι p , θέλουμε να υπολογίσουμε μια εκτίμηση $\hat{p}$ του p ώστε $\Pr[|\hat{p} - p| \leq \varepsilon p] > 1 - \delta$, για δεδομένα $\varepsilon, \delta \in (0, 1)$. Για τη δημοσκόπηση, θα ρωτήσουμε N πολίτες, που επιλέγονται ισοπίθانا και ανεξάρτητα από το σύνολο των πολιτών. Η εκτίμησης μας $\hat{p}$ θα είναι το ποσοστό των N πολιτών που τάσσονται υπέρ της μεταβολής. Χρησιμοποιώντας Chernoff-Hoeffding bounds, να υπολογίσετε (ως συνάρτηση των ε, δ , και p) το ελάχιστο μέγεθος N του δείγματος που χρειαζόμαστε. Βρείτε την τιμή του N για $\varepsilon = 0.02$ και $\delta = 0.05$, αν γνωρίζουμε ότι $p \in [0.1, 0.7]$ (και δείτε ότι αυτή η τιμή είναι ανεξάρτητη του πληθυσμού της χώρας!). Να υπολογίσετε ακόμη το ελάχιστο μέγεθος N' δείγματος (ως συνάρτηση των ε και δ) ώστε η εκτίμησης μας $\hat{p}'$ να ικανοποιεί $\Pr[|\hat{p}' - p| \leq \varepsilon] > 1 - \delta$. Ποια είναι η τιμή του N' για $\varepsilon = 0.02$ και $\delta = 0.05$;

Σημείωση: Πρόκειται για παραλλαγή της Άσκησης 4.5 του M. Mitzenmacher and E. Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.

Λύση

Πρώτη εκτίμηση

Έστω $X_1, \dots, X_N$ ανεξάρτητες δοκιμές Bernoulli τέτοιες ώστε $\Pr(X_i) = \mathbf{E}[X_i] = p$.

Οι μεταβλητές αυτές παίρνουν τιμή 1 όταν ένας πολίτης απαντάει “Ναι” με πιθανότητα p και τιμή 0 όταν απαντάει “Όχι” (με πιθανότητα $1 - p$).

Προφανώς $\hat{p} = \frac{1}{N} X$.

Έστω $X := \sum_{i=1}^N X_i$.

Γνωρίζουμε ότι για $0 < \varepsilon < 1$ ισχύει ότι (Chernoff Bounds):

$$\Pr(|X - \mathbf{E}[X]| \geq \varepsilon \mathbf{E}[X]) \leq 2e^{-\mathbf{E}[X]\varepsilon^2/3}.$$

Αφού $\mathbf{E}[X] = Np$

$$\Pr(|X - \mathbf{E}[X]| \geq \varepsilon \mathbf{E}[X]) \leq 2e^{-\mathbf{E}[X]\varepsilon^2/3}$$

$$\Pr(|X - Np| \geq \varepsilon Np) \leq 2e^{-Np\varepsilon^2/3}$$

$$\Pr(|\hat{p} - p| \geq \varepsilon p) \leq 2e^{-Np\varepsilon^2/3}$$

Θέλουμε $\Pr[|\hat{p} - p| \leq \varepsilon p] > 1 - \delta$,

Συνεπώς $2e^{-Np\varepsilon^2/3} \leq \delta$.

Λύνουμε ως προς N και παίρνουμε ότι:

$$N \geq \frac{3}{p\varepsilon^2} \ln\left(\frac{\delta}{2}\right)$$

Αριθμητική Εφαρμογή:

Στη χειρότερη περίπτωση όπου $p = p_{\min} = 0.1$ έχουμε

$$N \geq \frac{3}{(0.02)^2 \cdot 0.1} \ln\left(\frac{2}{0.05}\right) \approx 276666,7 \implies N = 276667.$$

Το αποτέλεσμα αυτό είναι ανεξάρτητο από το πλήθος του πληθυσμού, καθώς βασίζεται σε αδροίσματα ανεξάρτητων δειγμάτων Bernoulli.

Δεύτερη εκτίμηση

Ζητείται το ελάχιστο μέγεθος N' δείγματος (ως συνάρτηση των ε και δ) ώστε η εκτίμησή μας $\hat{p}'$ να ικανοποιεί $\Pr[|\hat{p}' - p| \leq \varepsilon] > 1 - \delta$.

Θα χρησιμοποιήσουμε ειδική περίπτωση για ανεξάρτητες 0/1 δοκιμές Bernoulli της προσθετική μορφή (*additive form*) των Chernoff-Hoeffding bounds.

Έστω ανεξάρτητες τυχαίες μεταβλητές X_i με $a_i \leq X_i \leq b_i$ και $S_n = \sum_{i=1}^n X_i$. Τότε:

$$\Pr(S_n - \mathbb{E}[S_n] \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right),$$

και το δύο-όψεων bound:

$$\Pr(|S_n - \mathbb{E}[S_n]| \geq t) \leq 2 \exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right).$$

Θεωρούμε τώρα την ειδική περίπτωση όπου οι $X_i, i \in [N]$ ακολουθούν κατανομή Bernoulli με πιθανότητα (παραμέτρο) p και καταλήγουμε στην εξής σχέση:

$$\Pr[|\hat{p}' - p| > \varepsilon] \leq 2e^{-2N'\varepsilon^2}$$

Θα υπολογίσουμε τον ελάχιστο αριθμό δειγμάτων N' , ώστε η εκτίμησή μας να ικανοποιεί:

$$\Pr[|\hat{p}' - p| \leq \varepsilon] > 1 - \delta$$

Επομένως θέτοντας $2 \exp(-2N'\varepsilon^2) \leq \delta$ και λύνοντας ως προς N' , έχουμε:

$$N' \geq \frac{1}{2\varepsilon^2} \ln\left(\frac{2}{\delta}\right), \quad N' = \left\lceil \frac{1}{2\varepsilon^2} \ln\left(\frac{2}{\delta}\right) \right\rceil.$$

Αριθμητική εφαρμογή:

$$N' \geq \frac{1}{2 \cdot 0.02^2} \ln\left(\frac{2}{0.05}\right) = \frac{1}{0.0008} \ln(40) \approx 1250 \cdot 3.6889 \approx 4611.1$$

Άρα:

$$N'_{\min} = \lceil 4611.1 \rceil = 4612$$

Για να έχουμε πιθανότητα τουλάχιστον 95% ότι το σφάλμα της εκτίμησης είναι μικρότερο από 0.02, απαιτούνται τουλάχιστον 4612 ανεξάρτητα δείγματα.

Άσκηση 9: (Capacited Max k -Cut)

Εκφώνηση

Στο Max k -Cut πρόβλημα δίνεται απλό μη κατευθυνόμενο γράφημα $G(V, E, w)$, με θετικά ακέραια βάρη $w : E \rightarrow \mathbb{N}$ στις ακμές, και ζητείται μια διαμέριση των κορυφών σε $k \geq 2$ σύνολα $(S_1, \dots, S_k)$, με $\cup_{i=1}^k S_i = V$, ώστε να μεγιστοποιηθεί το συνολικό βάρος των ακμών με άκρα σε διαφορετικά σύνολα. Ειδικότερα, αν συμβολίσουμε με

$$\delta(S_1, \dots, S_k) = \{e = \{u, v\} \in E : v \in S_i, u \in S_j \text{ και } i \neq j\}$$

το σύνολο των ακμών στο k -cut $(S_1, \dots, S_k)$, ζητείται να μεγιστοποιήσουμε το

$$w(S_1, \dots, S_k) = \sum_{e \in \delta(S_1, \dots, S_k)} w(e).$$

Θεωρούμε παραλλαγή του προβλήματος Max k -Cut όπου τα επιθυμητά μεγέθη $(c_1, \dots, c_k)$, με $c_1 + \dots + c_k = |V|$ των συνόλων που ορίζουν το k -cut είναι δεδομένα. Εφαρμόζουμε τον πιθανοτικό αλγόριθμο που τοποθετεί κάθε κορυφή v στο σύνολο S_i τυχαία, με πιθανότητα $c_i/|V|$, και ανεξάρτητα.

Να υπολογίσετε την αναμενόμενη τιμή του $w(S_1, \dots, S_k)$ (και να διερευνήσετε πότε μεγιστοποιείται). Να υπολογίσετε ακόμη άνω και κάτω φράγματα (ώστε να ισχύουν με μεγάλη πιθανότητα) για τα μεγέθη των συνόλων που προκύπτουν.

Λύση

1. Αναμενόμενη τιμή του βάρους του τυχαίου k -cut.

Έστω $n = |V|$ και $p_i = \frac{c_i}{n}$. Για κάθε ακμή $e = \{u, v\}$, ορίζουμε

$$X_e = \begin{cases} 1, & \text{αν } u \text{ και } v \text{ ανήκουν σε διαφορετικά σύνολα,} \\ 0, & \text{αλλιώς.} \end{cases}$$

Τότε

$$w(S_1, \dots, S_k) = \sum_{e \in E} w(e) X_e \implies \mathbb{E}[w(S_1, \dots, S_k)] = \sum_{e \in E} w(e) \Pr[X_e = 1].$$

Επειδή

$$\Pr[X_e = 1] = 1 - \sum_{i=1}^k \Pr[u, v \in S_i] = 1 - \sum_{i=1}^k p_i^2,$$

προκύπτει

$$\mathbb{E}[w(S_1, \dots, S_k)] = \left(1 - \sum_{i=1}^k p_i^2\right) \sum_{e \in E} w(e) = \left(1 - \sum_{i=1}^k \frac{c_i^2}{n^2}\right) \sum_{e \in E} w(e).$$

Σημείωση: Αυτή η τιμή είναι μέγιστη όταν $p_i = \frac{1}{k}$ για όλα τα i (δηλαδή $c_i = n/k$).

2. Άνω και κάτω φράγματα για τα μεγέθη των συνόλων.

Έστω για κάθε $i = 1, \dots, k$ και κάθε κορυφή $v \in V$ η Bernoulli-μεταβλητή

$$X_v^{(i)} = \begin{cases} 1, & \text{αν } v \in S_i, \\ 0, & \text{αλλιώς.} \end{cases}$$

Τότε

$$|S_i| = \sum_{v \in V} X_v^{(i)}, \quad \mu_i := \mathbb{E}[|S_i|] = \sum_{v \in V} p_i = n p_i = c_i,$$

όπου $p_i = c_i/n$.

Για ευκολία θα χρησιμοποιήσουμε το διπλό φράγμα (αλλιώς θα έπρεπε να χρησιμοποιήσουμε το άνω και το κάτω ξεχωριστά).

Για $0 < \varepsilon \leq 1$ και κάθε i ισχύει:

$$\Pr[||S_i| - \mu_i| \geq \varepsilon \mu_i] \leq 2 \exp\left(-\frac{\varepsilon^2 \mu_i}{3}\right).$$

Για $t > 0$, από το παραπάνω φράγμα προκύπτει ότι:

$$2 \exp\left(-\frac{t^2}{3\mu_i}\right) \leq \delta \quad \Rightarrow \quad t \geq \sqrt{3\mu_i \ln \frac{2}{\delta}}.$$

Άρα με πιθανότητα τουλάχιστον $1 - \delta$:

$$c_i - \sqrt{3c_i \ln \frac{2}{\delta}} \leq |S_i| \leq c_i + \sqrt{3c_i \ln \frac{2}{\delta}}.$$